Nova-HEI

Nova-Harness Embodied Intelligent

Abstract

Embodied applications must turn operational requirements into sustained, verifiable physical actions. However, tight coupling between task logic, models, hardware, and environment rules increases the cost of adding capabilities and adapting deployments. We propose **Nova-Harness Embodied Intelligent (Nova-HEI)**, a Harness framework designed for embodied applications with multiple robots and tasks, using shared task definitions, skill interfaces, and execution feedback. As a first step, we focus on vision-and-language navigation (VLN) for quadruped robots and develop **Nova-N**, a navigation prototype for studying closed-loop execution of long tasks with unknown target locations and reuse across deployments. The system combines active exploration, target pursuit, execution monitoring, and completion checks in a cycle of observation, decision, execution, review, and memory update. Traceable evidence memory within each task supports decisions across stages. Failure constraints and checks based on current observations help address repeated exploration, lost targets, and cases where a navigation stop is mistaken for task completion. The system also separates task logic, model protocols, and robot execution while leaving the internal parameters and inference process of existing VLN models unchanged. Shared interfaces and modular adapters allow the core workflow to be reused. The current prototype integrates LightNav and supports three types of backends: a physical Go2 robot, MuJoCo simulation, and benchmark evaluation. This implementation provides a basis for evaluating reliability in long tasks and the cost of adapting the system across deployments.

1 Introduction

Embodied applications, such as industrial inspection, campus maintenance, and indoor services, require robots to turn operational requirements into physical actions that can be carried out and checked over time. A complete workflow often includes navigation, search, inspection, manipulation, and interaction. An inspection workflow, for example, may use a drone for a wide-area survey, a quadruped robot for close inspection, and a quadruped robot with an arm for follow-up manipulation. Each platform contributes a different capability, but the workflow still needs a common way to describe goals, coordinate skills, and check results.

For developers and deployment teams, three issues make such workflows difficult to build. **Long tasks need clear dependencies and completion criteria**: a local action may stop before the overall goal is reached. **Models and robots differ in their interfaces and capabilities**: sensing, motion, and control protocols must be matched to task intent. **Robot teams need shared task state**: progress and evidence must be passed between stages so that robots can coordinate their work. When these requirements are built directly into task code tied to particular models or robots, adding capabilities or changing environments requires repeated integration and validation. So that a common runtime that manages tasks and feedback through clear interfaces.

To address these needs, we propose **Nova-Harness Embodied Intelligent (Nova-HEI)**, a Harness framework designed to make embodied applications easier to develop and deploy. Nova-HEI places a reusable task runtime between model capabilities and robot execution, using shared

task definitions, skill interfaces, and execution feedback to organize task state, action constraints, and result checks. Its broader design covers multiple tasks and robot platforms. Figure 1 shows how these application needs lead to the framework and how **Nova-N** implements its navigation branch. This paper focuses on target search and approach with a single quadruped robot; drones, quadrupeds with arms, and coordinated multi-robot workflows are part of the intended extension.

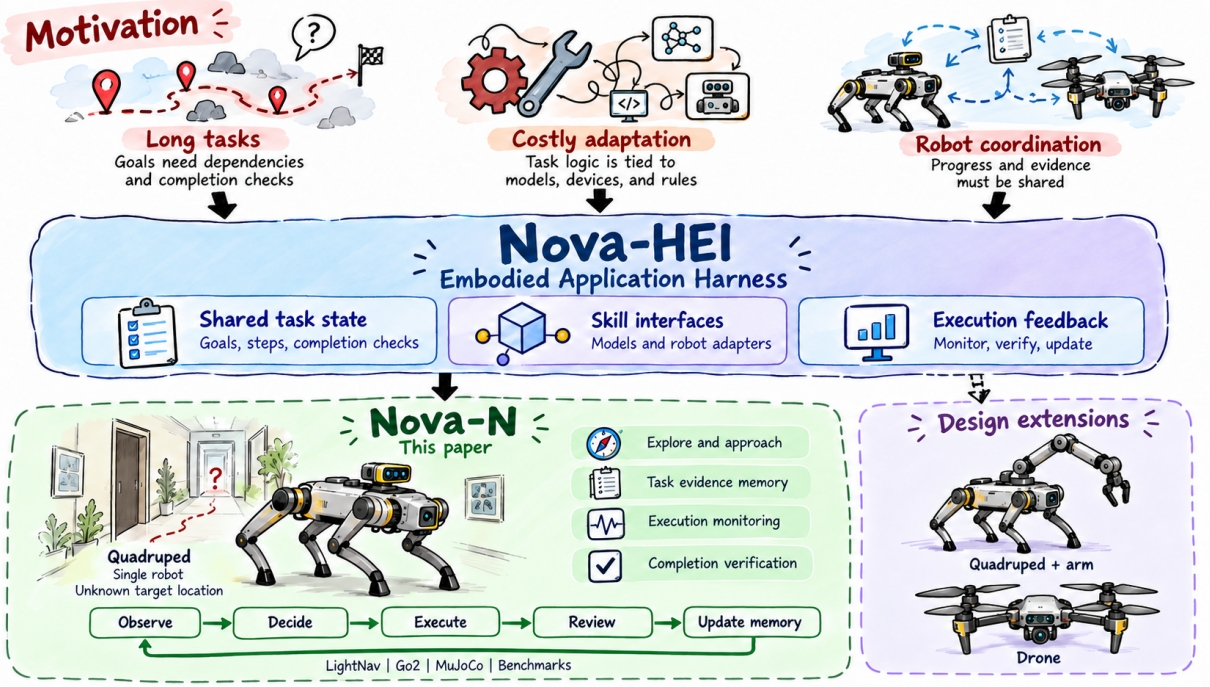


Figure 1: From application needs to Nova-HEI and Nova-N. Long tasks, deployment differences, and coordination needs motivate a shared Harness for task state, skill interfaces, and execution feedback. Nova-N is highlighted inside the framework as this paper's prototype for navigation with a single quadruped robot. The dashed region marks planned extensions to drones and quadrupeds with arms.

We develop **Nova-N**, a Vision-and-Language Navigation (VLN) prototype of Nova-HEI, to study how the task loop can improve **navigation to initially unseen targets**. Here, an "unseen target" is a target that is outside the robot's field of view when the task begins and whose location is not provided. The robot explores based on a natural-language target description, continues approaching once it finds relevant clues, and checks whether the task completion criteria have been met. This process brings together several key challenges: observing only part of the environment, maintaining state across task stages, handling differences between planned and actual motion, and deciding when the task is complete. It therefore provides a suitable first test of the framework.

Nova-N organizes the task into a repeated cycle of observation, decision, execution, review, and memory update. The system discovers targets through active observation and exploration of candidate routes or viewpoints. It records previous attempts and failures to limit repeated approaches, and keeps reliable target sightings to support continued pursuit after occlusion or changes in viewpoint. Memory within each task links spatial evidence, action history, and review results through shared observation references. It separates historical clues, actual execution feedback, and current judgments, giving later decisions a traceable basis. Execution monitoring detects problems such as blocked movement and lack of progress. After the underlying navigation component stops, the system continues to review task progress and uses the current observation to check whether the completion criteria have been met.

Following Nova-HEI's goal of ease of use, Nova-N uses a modular structure to make navigation capabilities easier to integrate and reuse. It separates task logic, navigation coordination, model protocols, and robot execution. Differences between deployments are handled through

configuration and adapter modules. Existing compatible combinations can be selected through configuration, while new navigation models or robot platforms can be added through clear interfaces. The current implementation requires no changes to the internal parameters or inference process of an existing VLN model. This allows improvements to the task loop to reuse existing navigation and control components and reduces changes to core task logic when the deployment setup changes.

The main work on Nova-N at this stage includes:

1. **A task loop for navigation to initially unseen targets.** Active exploration, continued target pursuit, execution monitoring, failure recovery, and completion checks are combined in one workflow. This helps existing VLN systems continue working toward a task goal when the target is not visible at the start.
2. **Evidence memory for decisions across task stages.** Observations, target clues, action results, and later reviews are linked to provide a traceable basis for exploration choices, finding a lost target again, and judging progress.
3. **Modular integration designed for ease of use.** Shared interfaces and configuration-based assembly allow the task core to be reused across environments, navigation models, and robot platforms, reducing the work needed for integration and deployment validation.

The current prototype focuses on target search and approach. It integrates LightNav and includes three types of backends: a physical Go2 robot, MuJoCo simulation, and benchmark evaluation. Evaluation will focus on task completion rates and the effectiveness of the task loop for navigation to initially unseen targets. Baseline comparisons and ablation studies will examine the contribution of each mechanism. We will also assess improvements across environments, backends, and VLN models. The value of the modular design will be evaluated through the amount of new code, the scope of changes to core logic, and the work required for integration validation. These results will support later extensions to multiple tasks and collaboration among multiple robots.

2 Related Work

2.1 VLN

Vision-and-language navigation maps language instructions and visual observations to navigation actions. Recent work on real robots has mainly improved reliability and deployment efficiency through hierarchical execution, long-term memory, and unified navigation models.

NaVILA (RSS 2025) [1] connects a vision-language model to a movement policy through language-based intermediate actions, enabling long-distance navigation and deployment across different legged robots. The authors report failures to correct deviations, showing that task recovery remains an issue in hierarchical execution. DualVLN (ICLR 2026) [2] combines low-frequency semantic planning with a high-frequency diffusion policy. It generates continuous trajectories from pixel goals and latent features to improve avoidance of moving obstacles. However, its local policy still has limits when the selected goal has a large error or refers to the wrong target. These methods focus on coordination between navigation decisions and movement execution, and transfer requires matching their policies or internal representations. We keep existing navigation and control components and study how explicit task state, execution feedback, and target checks can handle deviations during long tasks.

Modeling a long history requires a balance between retaining information and controlling computation. StreamVLN (ICRA 2026) [3] uses a sliding-window KV cache, slowly updated memory, and spatial token pruning to support streaming navigation with low latency. AstraNav-Memory

(CVPR 2026) [4] uses visual compression to store more history and support navigation memory across tasks. Its ablation analysis [4] also shows that excessive compression and overly long context can reduce performance. These methods improve memory and reasoning inside the model, so integration depends on their encoding, caching, or training mechanisms. Nova-N instead links observations, target sightings, and actual action results at the task level. It selects relevant evidence to support decisions across stages, and its current memory is limited to a single task.

NavFoM (ICLR 2026) [5] combines different navigation capabilities through joint training across tasks and robot platforms. It uses time and viewpoint labels, along with sampling that accounts for the available budget, to handle different observation setups. Its results demonstrate the potential for generalization at the model level, while practical deployment still requires connections to each platform's planning and control components. We focus on runtime issues beyond a unified model, including execution monitoring, target confirmation, and interface adaptation. Models such as NavFoM can provide the underlying navigation capability, but model generalization and task workflow reuse need separate evaluation.

2.2 VLN Agents

VLN agents organize long tasks through skill calls, memory, and feedback, allowing local navigation to serve an ongoing task goal. Previous work already covers active exploration, self-correction, and completion verification. The differences in our work therefore need to be established through how evidence is organized, how execution is constrained, and what integration requires.

AgentVLN (ECCV 2026) [6] is the work most directly related to ours. It uses a vision-language model to select perception and planning skills. Three-dimensional path projection, active exploration, and geometric queries made as needed support navigation and target localization. Its approach combines depth, pose, SLAM, and dedicated instruction tuning, and reports deployment on local devices. These requirements should be evaluated separately from computational efficiency; see its method and deployment details [6]. Nova-N currently calls an existing VLN model through external interfaces. At the upper level, it explicitly maintains coverage of observed directions, candidate failures, target pursuit, and checks based on current observations. It does not require access to internal VLN features or changes to the model's inference process. This difference offers a way to reuse existing systems. Its reliability and cost benefits still need to be compared with AgentVLN under comparable conditions.

ABot-AgentOS (2026) [7] combines planning, isolated skill execution, verification at multiple stages, and persistent graph memory in a general robot runtime layer. It supports interaction across tasks and execution across robot platforms. Results on an initial benchmark subset support the value of runtime improvements, but are not enough to establish the full costs and benefits under different real deployment conditions. We focus on target search and approach. Observation references connect spatial evidence, action results, and later reviews within a task. The current system does not include graph memory across tasks or automatic self-improvement. With this focused scope, Nova-N addresses reliability during long tasks through state transitions, failure constraints, and completion criteria that can be inspected.

Overall, existing work provides effective navigation models and agent mechanisms. Nova-N further studies a reusable task loop built on existing systems. It uses evidence memory and execution review to improve extended target search while keeping the navigation model and movement execution components unchanged. Clear interfaces separate the task workflow from differences between models and robot platforms.

3 Agent Framework

Nova-N provides a task-level control loop for searching for and approaching targets that are initially out of view and whose locations are unknown. It leaves the parameters and inference process of the existing vision-and-language navigation model unchanged. Section 3.1 outlines the task loop, and Section 3.2 explains its core mechanisms in the order in which a task runs, from start to finish.

3.1 Overall Workflow

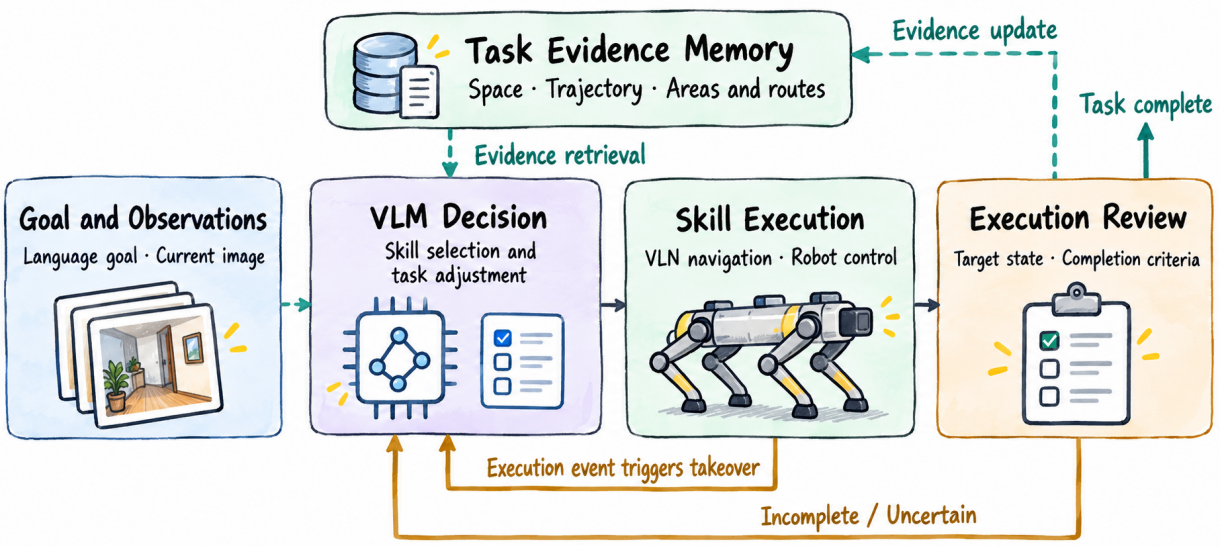


Figure 2: The overall Nova-N task loop. The VLM combines the goal, current observation, and evidence memory to select skills, then adjusts the task based on execution events and review results. Relevant evidence is stored in memory within the task, and the task ends when its completion criteria are met. Solid lines show execution and feedback; dashed lines show evidence exchange.

Given a natural-language goal and a task budget, Nova-N advances the task through a cycle of observation, decision, execution, review, and memory update. The upper-level agent organizes evidence, selects skills, and maintains task state. The underlying VLN model generates local paths, while the robot execution layer tracks those paths and returns feedback. During navigation, upper-level decisions are mainly triggered by feedback events. Current observations and historical memory jointly support subsequent adjustments.

A navigation stop or a detection provides a clue that still needs review. The system uses current evidence to assess target state and task completion, records action history and review results in memory, and decides whether to continue exploration, maintain target pursuit, or end the task. Figure 2 summarizes this loop, and Figure 3 shows its operation in more detail.

3.2 Core Mechanisms of the Task Loop

3.2.1 Task Initialization and Component Assembly

The task takes a natural-language goal g and a budget B as input. The system loads skill descriptions and environment constraints, creates the initial task state and task memory, and uses application configuration to assemble the model client, navigation session, and robot execution components. Environment rules are expressed through configuration, prompts, and deployment validators. Dedicated adapters handle differences in model protocols and robot platforms.

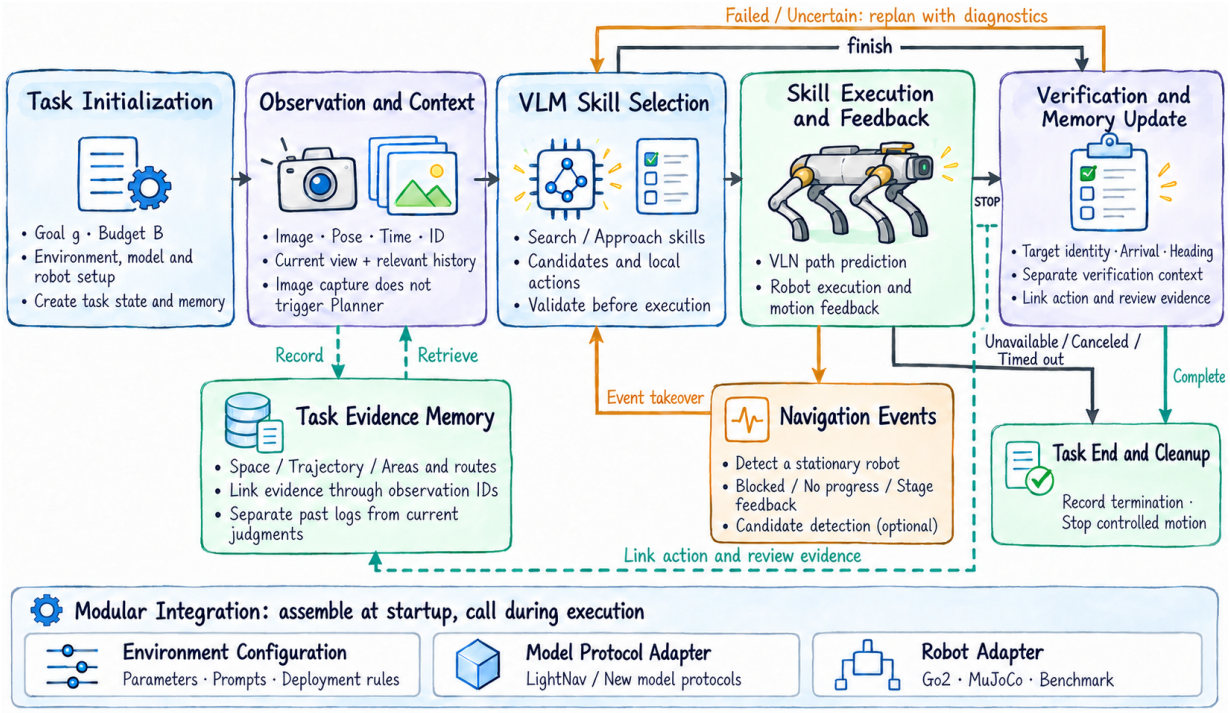


Figure 3: Nova-N's task core and modular integration. The main workflow covers task initialization, observation and evidence organization, VLM decisions, skill execution, and completion verification, corresponding to Sections 3.2.1–3.2.5. Execution events trigger upper-level control. If verification fails, its diagnostic feedback is passed back to the decision stage. Action and review evidence is linked and stored in memory. The system performs task cleanup when completion is confirmed or another termination condition is met. The adaptation layer at the bottom is assembled during initialization and provides shared interfaces during execution.

Shared interfaces define observations, predictions, actions, feedback, and component lifecycles so that task logic can be reused. Existing compatible combinations can be selected through configuration. Adding a model or robot requires implementing the relevant interfaces and validating compatibility. LightNav is the current built-in model, and the native Go2 service still uses the default LightNav setup. Other models require the corresponding protocol and server support. The adaptation layer at the bottom of Figure 3 represents the interface support available throughout the task.

3.2.2 Observation Capture and Evidence Context

Each observation is written as $o_i = (I_i, p_i, \tau_i, \mathrm{id}_i)$, containing an image, the robot pose at image capture, a timestamp, and a stable observation ID. Images captured once per second during navigation are collected as passive evidence; image capture alone does not trigger the Planner. When an upper-level decision or verification is needed, the system selects the current observation and gathers historical evidence relevant to the task.

Task memory $\mathcal{M}$ contains three types of records. **Spatial Memory** stores images, poses, semantic reports, and target sightings to support retrieval of relevant observations and identity checks. An observation location cannot be treated directly as an object's coordinates. **Trajectory Memory** links action intent, actual motion, and reviews before and after an action. It helps identify repeated attempts and returns to previously visited locations; missing intermediate routes are not inferred from their start and end points. **Topology Memory (Areas and Routes)** records area identity, previously used entrances, and routes that still need inspection. Connections between areas require support from both actual motion and visual review.

Before the kth upper-level decision, the system retrieves context using the goal, task state s_k, current observation, and event diagnostics e_k:

$$\mathcal{C}_k = Retrieve(\mathcal{M}; g, s_k, o_{i_k}, e_k).$$

The system applies separate limits to retrieved observations, routes, and action histories, and includes each original image only once in a context. When needed, it compares the current image captured while stopped with historical images of areas and entrances to support place matching. Historical sightings, execution feedback, and current visual judgments are recorded separately. An empty list of area candidates does not establish that the search is complete.

3.2.3 VLM Skill Selection and Event Scheduling

The upper-level VLM acts as the Planner. Using skill descriptions and tool specifications, it explicitly selects a skill z_k and an action a_k based on the current observation, task state, relevant evidence, and event diagnostics:

$$(z_k, a_k) = Planner(g, s_k, o_{i_k}, \mathcal{C}_k, e_k).$$

The `search_navigation` skill supports exploration when no specific target clues are available. The `approach_target` skill supports approaching and checking a clear or possible target, as well as searching nearby to find it again. Both skills share candidates, exploration objectives, and target pursuit state. Common observation, turning, and recovery actions do not have to specify a navigation skill.

During navigation, the Planner is called mainly in response to feedback events, rather than at fixed intervals. `DIRECT`, `STAGING`, and `VERIFY` denote the underlying execution, upper-level control, and completion verification states in this decision process. Table 1 gives the scheduling rules.

Table 1: Events and scheduling rules for VLM decisions.

@>p0.18>p0.33>X@ **Event** **Trigger** **Scheduling action**

Underlying navigation stops LightNav outputs `STOP`. Enter `VERIFY` to check target identity, arrival, and heading.

Verification fails Any completion criterion is false or uncertain. Enter `STAGING` and replan using the verification diagnostics.

Candidate target detected Two consecutive frames have the same category and matching detection boxes. Interrupt navigation and enter `STAGING` to review the candidate.

Robot remains stationary Displacement is below 0.05 m and accumulated rotation is below 10° over about 8 s. Interrupt navigation and enter `STAGING` to inspect the cause.

Execution problem or stage ends The execution interface reports blocked movement, no progress, the end of a local observation segment's budget, or similar feedback. Enter `STAGING` to decide whether to continue or adjust the strategy.

Adjustment complete; return to execution `look` or `orient` succeeds, and the Planner supplies the remaining instruction. Return to `DIRECT` and resume LightNav navigation.

Before an action is submitted, the system checks its output structure, skill requirements, current evidence, and environment rules. Validation runs on a copy of the state, so rejected actions do not change the actual execution state. Navigation choices must refer to a currently visible candidate. Exploration candidates must also state what observation question is expected to be answered by moving closer.

3.2.4 Skill Execution and Event Feedback

The tool layer executes validated actions. The `navigate` tool calls the VLN model to generate local paths. The `look`, `orient`, and `recover` tools use the robot's observation, turning, and recovery capabilities. The navigation coordination layer manages sessions, receives feedback, and handles execution interruptions. The robot execution layer handles path tracking and motion constraints. Each predicted path is paired with the original image capture timestamp and pose. It is interpreted in the robot's body frame at capture time, then transformed by the executor. Predictions without a matching pose are not executed.

Pose monitoring actively detects when the robot remains stationary. Blocked movement, lack of progress, and the end of a local observation segment's budget are handled based on feedback actually returned by the execution interface. The current system does not continuously detect semantic progress. The Planner uses these events and the current observation to assess the cause, then decides whether to gather more observations, adjust the strategy, or call a recovery action supported by the backend. After `look` or `orient` succeeds and the Planner supplies the remaining instruction, control returns to LightNav.

YOLO-World detection during search navigation can be enabled through configuration. `TargetWatch` runs asynchronous detection on images captured along the route. When two consecutive frames have the same category and matching detection boxes, the system checks image freshness, session state, and whether the result belongs to the current navigation action, then interrupts navigation. The Planner reviews the candidate using the triggering image, detection boxes, and the current observation captured while stopped. It then decides whether to continue searching or start target approach. The system briefly suppresses rejected candidates and prevents late results from affecting the current state. Detection is disabled during target approach, turning, and recovery. This module is off by default and requires streaming observations and a navigation interruption interface. If it is unavailable, the system continues basic navigation and handles other events.

3.2.5 Completion Verification, Memory Update, and Task Termination

After execution results return, the system links action intent, actual motion, and subsequent review, then updates candidate, target, and area states. Candidate failures are counted by observation location. The first and most recent reliable target sightings support continued pursuit. When a target is temporarily lost, the robot can make a limited number of attempts to find it through directed observation or currently visible viewpoints. It can defer pursuit and resume exploration when needed. Updated memory is retrieved for the next decision, linking evidence across task stages.

A LightNav `STOP` or an upper-level `finish` proposal triggers completion verification, which checks target identity, arrival, and heading. By default, verification uses a separate request through the same VLM client, providing only the target requirements and the current image captured while stopped. This separates verification from the historical decision context. If any completion criterion is false or uncertain, the system returns to `STAGING` with diagnostic feedback. Once the task requirements are confirmed, it records completion and ends the task. It does not declare success if verification encounters an error. Internal verification supports task scheduling; experimental outcomes are still determined by independent scoring.

The task can also end when the agent gives up, the budget is exhausted, or the task is canceled. Navigation service errors, cancellation, and timeouts are treated as unavailable execution, and the task lifecycle performs the corresponding cleanup. In contrast, the end of a local observation segment's budget is passed to the Planner as stage feedback. When an action ends or exits because of an error, the backend stops motion controlled through that connection.

4 Experiments

1. **Experimental Setup:** Task definition, environment splits, baselines, independent scoring, budgets, and statistical protocol.
2. **Long-Horizon Navigation:** Overall results and results grouped by task length, with the agent enabled and disabled, to directly address the main motivation.
   (LightNav)

   1. Validation across platforms
      Benchmark results:

      - Insight-Habitat-GS: compare results with and without the agent.
      - R2R-CE: compare results with and without the agent.
      - RxR-CE: compare results with and without the agent.
      - R2R-PE: compare results with and without the agent.
      - HM3D-ObjectNav-v2: compare results with and without the agent. ★

      MuJoCo results: compare results with and without the agent.
      Go2 results: compare results with and without the agent in a single environment.
   2. Validation across environments
      Go2 results in multiple environments, including offices, test areas, and outdoor campuses.
   3. Compatibility with multiple VLN frameworks, evaluated through benchmarks. Assess both the improvement provided by the agent and the integration cost for each VLN model.

      - LightNav results
      - InterVLN results
3. **Ablation Studies:** Explain the contributions of memory, exploration, maintaining target identity, failure constraints, and completion checks.
4. **Cross-Scenario and Cross-Platform Evaluation:** Test the effectiveness of a fixed core workflow under different deployment conditions.

   Section 1: Benchmarks / Section 2: Real-robot experiments

   Agent–VLN comparison / Ablation studies: use one benchmark, Insight-Habitat-GS.

5 Conclusion

We presented Nova-HEI as a Harness framework for organizing embodied tasks through shared task definitions, skill interfaces, and execution feedback. Its navigation prototype, Nova-N, turns target search and approach into a task loop that links observations, decisions, actions, review, and memory updates. By connecting target sightings and actual action results through observation references, the system supports decisions across stages while keeping past clues separate from current evidence. Explicit failure constraints, execution monitoring, and separate completion checks provide a structured way to handle repeated search, lost targets, and premature stops.

Nova-N also separates task logic from model protocols and robot execution, allowing the core workflow to reuse existing navigation and control components without changing VLN model parameters or inference procedures. The current implementation integrates LightNav and provides backends for the Go2 robot, MuJoCo simulation, and benchmark evaluation. This establishes an implementation basis for evaluating task reliability and adaptation effort across deployments. The present work remains focused on single-robot navigation; extending the same runtime to manipulation, drones, and coordinated robot teams is a next step toward complete embodied workflows.

6 Future Work

Nova-HEI currently uses Nova-N as its first implementation, focusing on search for and approach to initially unseen targets with a single robot. Current integration mainly supports LightNav, and memory is limited to one task. Support for multiple tasks and coordination among multiple robots remains at the overall design stage. Future work will focus on task reliability, persistent memory, ease of deployment, and collaboration among different robot platforms. We will gradually test the framework in more complex embodied applications.

Improve the task loop under complex conditions. Current exploration, target pursuit, and completion checks depend on visual observations and model judgments. Their reliability under changing occlusion, interference from similar targets, and environmental changes still needs systematic evaluation. We will study checks that combine visual, geometric, and motion feedback, improve target identity tracking, fault diagnosis, and recovery strategies, and adjust observation frequency and exploration budgets based on observation uncertainty and task progress. Evaluation will cover task completion rate, false completion rate, recovery success rate, and additional inference cost to examine the relationship between reliability and execution efficiency.

Extend memory from one task to reuse across tasks. Current memory supports continued decision-making within one task, but does not yet accumulate or update experience across tasks. We will study ways to store information about areas, routes, targets, and failed attempts across tasks, so that past experience can be retrieved together with its supporting evidence. To handle information that becomes outdated as environments change, we will manage how long it remains valid, detect conflicts, and check past information again when needed. These mechanisms should allow historical experience to support current decisions. We will assess their benefits through exploration efficiency and cases of incorrect reuse in repeated tasks.

Improve integration for developers and deployment teams. We will expand support for navigation models, environments, and robot platforms. We will further clarify component inputs and outputs, capability requirements, lifecycles, and error feedback, and improve adapter templates, configuration checks, and compatibility validation tools. By recording component integration time, new code, the scope of changes to core logic, and deployment validation effort, we will evaluate improvements in development and deployment efficiency. This will turn ease of use from a design goal into measurable engineering outcomes.

Extend support to drones, robot arms, and other platforms. Building on ground navigation, we will explore how the shared task framework can support movement in three dimensions and physical manipulation. Drone support will focus on three-dimensional spatial representations, observation viewpoints, and flight constraints. Robot arm support will focus on object localization, reachability, contact feedback, and result verification. We will extend interfaces for skill preconditions, execution state, and completion criteria so that different robots can share task organization and evidence management. Each robot's execution module will handle its specific movement and manipulation requirements.

Extend single-robot navigation to multiple tasks and robot teams. We will first study

dependencies and transitions between skills such as navigation, inspection, and manipulation. We will then extend the framework to assign work across robots, pass evidence between them, and reassign work after failures under a shared task state. For example, a drone could identify an area with a possible problem, a quadruped robot could inspect it closely, and a platform with manipulation capabilities could carry out follow-up work. Validation will focus on end-to-end task completion rate, correct handoffs between stages, coordination overhead, and recovery from errors. These studies will gradually test Nova-HEI's ability to organize complete operational workflows.

Reference

[1] An-Chieh Cheng, Yandong Ji, Zhaojing Yang, et al. *NaVILA: Legged Robot Vision-Language-Action Model for Navigation*. Robotics: Science and Systems (RSS), 2025. Paper.

[2] Meng Wei, Chenyang Wan, Jiaqi Peng, et al. *Ground Slow, Move Fast: A Dual-System Foundation Model for Generalizable Vision-and-Language Navigation*. International Conference on Learning Representations (ICLR), 2026. Paper.

[3] Meng Wei, Chenyang Wan, Xiqian Yu, et al. *StreamVLN: Streaming Vision-and-Language Navigation via SlowFast Context Modeling*. IEEE International Conference on Robotics and Automation (ICRA), 2026. Paper.

[4] Botao Ren, Junjun Hu, Xinda Xue, et al. *AstraNav-Memory: Contexts Compression for Long Memory*. arXiv:2512.21627, 2025. Preprint. CVPR 2026 publication.

[5] Jiazhao Zhang, Anqi Li, Yunpeng Qi, et al. *Embodied Navigation Foundation Model*. International Conference on Learning Representations (ICLR), 2026. Paper.

[6] Zihao Xin, Wentong Li, Yixuan Jiang, et al. *AgentVLN: Towards Agentic Vision-and-Language Navigation*. European Conference on Computer Vision (ECCV), 2026. Paper. Official code.

[7] Jiayi Tian, Shiao Liu, Yuting Xu, et al. *ABot-AgentOS: A General Robotic Agent OS with Lifelong Multi-modal Memory*. arXiv:2607.10350, 2026. Paper.